

Measuring the Human-Agent Behavioral Gap in Game-Based CAPTCHAs

Jenny Chen, Coco Xu
{jennycjx, cocozxu}@stanford.edu

June 3, 2026

1 Background and Problem Setup

The failures of recognition-based CAPTCHAs. CAPTCHAs were introduced to tell whether a user is human, so that sites can deter bot attacks and spam. For two decades, the dominant CAPTCHAs asked users to recognize something a machine supposedly could not: reading a string of distorted, overlapping letters, or picking out every image tile containing a traffic light or a crosswalk. The design assumption was simple, that the task should be easy for a human but hard for a machine [Von Ahn et al., 2003]. That assumption no longer holds. Multimodal large language models now match or beat human accuracy on distorted-text and object-selection challenges, and agentic vision-language solvers generalize across widely deployed schemes [Teoh et al., 2025]. Automated traffic has even been reported to surpass human traffic on the web [Imperva, 2025]. Recognition alone can no longer separate humans from machines, which pushes CAPTCHA design toward probing *how* a task is solved rather than *whether* it is solved.

Two paradigms for human-agent separation. Defenses against this pressure fall into two broad families. The first, *behavioral biometrics*, looks at how the input is physically produced: mouse-dynamics systems model the kinematics and neuromotor structure of human cursor movement to flag synthetic trajectories [Shen et al., 2013, Acien et al., 2022, Khan and Hou, 2024]. This line rests on a mature body of motor-control theory: Fitts’ law for pointing [Fitts, 1954], the minimum-jerk model of arm movement [Flash and Hogan, 1985], the steering law for trajectory-constrained tasks [Accot and Zhai, 1997], and involuntary physiological tremor in the 8 to 12 Hz band [Elble and Koller, 1990]. Each of these predicts structure in human movement that a scripted path lacks. The strongest known attack on this channel is generative: DMTG trains a diffusion model to synthesize human-like mouse trajectories offline, yet it lowers detector accuracy by only a few points (95% to 90%), so a residual, detectable gap survives [Liu et al., 2024]. The second family, *cognitive challenge design*, looks

at what a player can perceive or reason about, using spatial, perceptual, or semantic tasks that remain hard for current models [Luo et al., 2025, Kharlamova et al., 2025, Ding et al., 2025]. Its premise follows dual-process accounts of cognition [Kahneman, 2011]: humans rapidly and automatically perceive animacy, causality, and social structure in moving displays [Heider and Simmel, 1944, Scholl and Tremoulet, 2000], whereas an agent has to reconstruct those judgments through explicit computation. The benchmark lineage here runs from OpenCaptchaWorld [Luo et al., 2025], the first platform to test multimodal LLM agents on dynamic puzzles, to NextGen-CAPTCHAs [Liu et al., 2026], which hardens many puzzle families against frontier agents. Both score a challenge on its final answer and limit the agent to clicking, dragging, and taking screenshots.

Robustness as the question. Neither paradigm is secure on its own. Vision and vision-language solvers now break widely deployed visual CAPTCHAs outright [Plesner et al., 2024]; reinforcement-learning agents have learned mouse and click behavior that defeats score-based behavioral detectors [Akrouit et al., 2019]; and cognitive challenges hardened enough to stop frontier models tend to frustrate humans as well [Ding et al., 2025]. Most of these studies, though, ask only whether a fixed detector can be fooled. What is less well understood is *which* behavioral signal actually carries the human-agent separation, and how much of that signal an agent can reproduce. That is the question we study. Our setup also differs from the closest prior work in two ways. Unlike DMTG, which generates traces offline, our adversary is a live agent that reads each game’s source and writes solver scripts in the loop. And unlike the cognitive benchmarks, which label a session by its final answer, we label a session by the trajectory it produced.

Problem setup. We take a single interaction session as input: the full stream of (x, y, t) mouse events a player produces while completing a short micro-game. The output is a binary label, human or agent, with a confidence score computed from the trajectory alone. A deployable game-based CAPTCHA has to satisfy five constraints at once: (i) *compleatability*, a human pass rate of at least 95% at a median completion under 10s; (ii) *separation*, a trajectory classifier that reaches AUC at least 0.85 against frontier agents; (iii) *robustness*, so an imitator trained on human trajectories cannot evade the classifier while still solving the game; (iv) *statelessness*, so each challenge is independently verifiable with no server-side state; and (v) *accessibility*, so solving does not depend only on fine motor precision or visual acuity.

We organize the study around two questions:

- **Q1.** Does trajectory information add separation beyond outcome alone?
- **Q2.** Is the gap robust to a basic imitation attack?

Our working hypothesis is that humans leave structured, continuous motor noise, physiological tremor, and real-time error correction, while an agent

that synthesizes events programmatically produces trajectories that are more uniform, faster, and free of any involuntary biological signal, whether or not it solves the game. The hypothesis is falsifiable: if an adaptive agent can drive the detector down to chance on tunnels it has not seen, the hypothesis fails.

2 Approach

Attacker model. The center of our method is the attacker we evaluate against. Rather than a fixed bot or an offline trajectory generator, we give a real coding agent (Claude Code) the full attacker surface a determined adversary would have: it drives the browser through Playwright, inspects the page source of each game, and writes and executes its own solver scripts. The agent therefore plays each game the way a person automating it would, by reading the implementation, reasoning about it, and iterating on a solver in a live browser.

2.1 Games

Trace-the-Tunnel (motor channel). The player presses a green start dot and drags to a red end dot along a curved tunnel on a 600×350 canvas without crossing the tunnel walls. Leaving the boundary auto-resets the same tunnel, and a shared pool of ten procedurally generated tunnels is reused across all sessions. The task takes humans about 1.3s and is meant to be easy: its purpose is not to be hard to solve but to elicit a dense, high-frequency record of *how* a cursor is driven through a constrained path. For this game the agent is allowed to inspect the source and execute arbitrary scripts.

Reasoning games (cognitive channel). We also built two reasoning games that target perception rather than motor control. In Pick-the-Personality, four dots move with distinct behavioral signatures (leader, follower, hyper, lazy), and the player clicks the one matching a named personality; because every dot behaves differently, an odd-one-out shortcut does not work. In Roll-or-Slide, four shapes move and the player clicks the one whose motion follows a different rule. Both rest on the premise that humans perceive animacy and personality in motion through fast, automatic processes [Kahneman, 2011, Heider and Simmel, 1944, Scholl and Tremoulet, 2000] that an agent must reconstruct by explicit computation, and both take a single-commit click as the response.

2.2 Trajectory representation

Logging. Trace-the-Tunnel logs every `mousemove`, `mousedown`, and `mouseup` event as (x, y, t) , timestamped with `performance.now()`, together with a per-event `inside_tunnel` ground-truth flag. We apply no throttling or polling, so inter-event intervals carry the timing signature of the underlying input pipeline.

Features and two-head split. We summarize each trajectory with 25 features in six theoretically motivated groups: kinematics [Flash and Hogan, 1985], curvature [Accot and Zhai, 1997], path geometry [Fitts, 1954], centerline deviation, tremor in the 8 to 12 Hz band [Elble and Koller, 1990], and inter-event timing. Rather than report a single number, we keep the first five groups (20 features) as a *motor* head and the timing group (5 features) as a *mechanism* head, because they capture independent kinds of evidence: how the hand moved versus how the events were produced. Splitting them lets us name which channel a given attack defeats. The classifier that consumes these features is described in Section 3.

2.3 Adversaries: how the agent attacks each game

Trace-the-Tunnel: model, dispatch, verify. On the motor game the agent writes a Playwright solver and refines it in a tight loop. It starts by picking a parametric model for the path. Most often this is a centerline fit from a small bank of human example traces (when we provide one), occasionally a cubic spline or a Bézier when no examples are given. It then iterates over the model’s hyperparameters: number of waypoints, dispatch interval, speed schedule, jitter amplitude, and so on. Each iteration runs the solver in a real browser, captures the resulting (x, y, t) trace from the page, and verifies it against two criteria. The hard criterion is success on the game itself: does the trajectory finish without crossing the tunnel walls. The soft criterion, when example traces are available, is qualitative similarity to those examples, judged by the agent reading the numbers in its own context. The loop stops when both criteria are satisfied to the agent’s own standard, which in practice means a handful of completed tunnels and a trace that “looks roughly human” to it. In Experiment 2 this loop runs unsupervised; in Experiment 3 we inject a specific tell to fix on each round (flat speed, no tremor, hugging the centerline), and the agent picks hyperparameters that target that tell.

Reasoning games: screenshot stack, characteristic analysis. On Pick-the-Personality and Roll-or-Slide the response is a single click, so there is no trajectory to refine. The relevant information is in motion across frames, and the agent cannot perceive video directly. Its behavior is to capture a stack of canvas screenshots over ten to fifteen seconds at a fixed interval, then write a Python script that extracts each object’s centroid in each frame and computes per-object characteristics: velocity magnitude and direction, total displacement, and pairwise lagged cross-correlations between objects. The agent then defines a numerical signature for each named label (“leader is the one whose motion precedes the others at positive lag,” “follower is the one with the highest single-pair lagged correlation,” and so on) and picks the object whose signature wins. The full chain is visible in the agent’s own writeups; when an early round is wrong, it revises one of these formulas and re-runs.

Reinforcement-learning agent. The RL attacker is a small actor-critic policy over a Python reimplementation of the tunnel, pretrained by behavioral cloning on the 172 completed human trajectories and fine-tuned with PPO. The reward combines arc-progress shaping and a completion bonus with an anti-detection penalty that pushes per-episode centerline deviation, mean speed, and path efficiency toward the human distributions. It dispatches events through the Chrome DevTools Protocol so its mechanism-head signature matches a real device’s.

3 Evaluation and Results

Setup. We collected 217 human sessions across six participants on the shared ten-tunnel pool, plus between 30 and 120 agent sessions per strategy. We treat the detector as holding when the combined AUC stays above 0.85 and as defeated when it falls below; $\text{AUC} \approx 0.5$ is the human-indistinguishable region. Everything runs on a single Apple-silicon MacBook Pro. The classifier and RL training are CPU-only PyTorch, and data collection uses Playwright-driven Chromium for the agent and stock Chromium on the participant laptops for the humans.

Detector. Our primary classifier is a Random Forest (200 trees, default depth, fixed seed 42, fit on `StandardScaler`-normalized features). We chose a random forest for three reasons. The features mix scales (centerline deviation in pixels, Δt in milliseconds, tremor power in spectral units), and trees do not need careful rescaling to compare them. The model exposes per-feature importances we can read off, which is what lets us name which feature each attack defeats. And tree ensembles do not smooth small structured tells the way a linear model would. We sanity-check it against two alternatives. Logistic regression and gradient boosting both classify the same feature vectors and reach AUC 0.999 and 0.984 respectively against the visual baseline, compared to 1.000 for the random forest. As a feature-free check, a 1D-CNN over the raw event sequence reaches AUC 0.985 on the same comparison, which tells us the signal does not depend on our feature choices. Cross-validation is 5-fold `StratifiedKFold` for the visual, LLM-solver, and RL attacks; for replay-family attacks we switch to 5-fold `GroupKFold` grouped by source-human session so the same source bank never appears in both train and test.

3.1 Experiment 1: Replay, is the delivery channel faithful?

Question. If we hand the agent a real human trajectory, can it deliver that trajectory back indistinguishably? This isolates delivery from generation: if replayed traces are still detectable, the channel itself leaks and nothing downstream can be trusted.

Dispatch iteration	Motor AUC	Mechanism AUC	Combined AUC
Iter 1: CDP	0.71	0.77	0.82
Iter 2: HID	0.46	0.69	0.63
Iter 3: init-forge	0.28	0.54	0.36

Table 1: Replay fidelity across dispatch mechanisms. As delivery is refined, replayed human traces fall into the human-indistinguishable region (AUC $\approx$ 0.5).

Result. Three iterations of progressively lower-stack dispatch close the gap (Table 1). The first iteration uses the *Chrome DevTools Protocol (CDP)*, which dispatches events from the browser process instead of from page JavaScript. This removes the `setTimeout` timing floor and the `isTrusted=false` flag that an in-page dispatcher leaves behind. But the renderer-IPC hop adds sub-millisecond timing noise, and the detector amplifies that noise through derivatives like jerk and tremor power, so combined AUC only falls to 0.82. The second iteration leaves Chromium entirely and injects events at the OS-level *Human Interface Device (HID)* stream, the same path a real USB mouse takes. Timing artifacts collapse, but the HID API rounds coordinates to integer screen pixels and the macOS event coalescer occasionally merges rapid moves; the classifier shifts onto that spatial quantization, and combined AUC reaches 0.63. Each step lower removes one artifact and reveals another, so the third iteration stops fixing the channel and sidesteps it: an init script injected before `game.js` loads forges the Web APIs the captcha actually reads, overriding `MouseEvent.prototype.clientX`, `clientY`, and `performance.now()` to return the source trace’s values directly. CDP still fires events to advance the game’s state machine, but every feature the captcha records is now bit-identical to the source human trace, and combined AUC ends at 0.36, below the chance line.

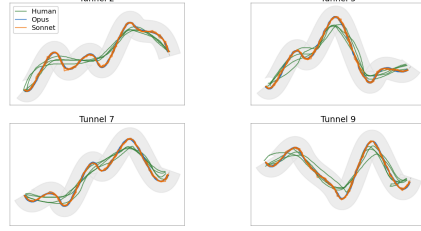
Takeaway. No userspace dispatch channel is fully leak-free: CDP leaks renderer-IPC timing, HID leaks spatial quantization, and the arms race only ends once we stop delivering through a channel at all and forge the values the captcha reads. With that escalation the delivery channel is faithful, so any separation from here on is about *generation*, not delivery.

3.2 Experiment 2: Few-shot generation on unseen tunnels

Question. Now that the agent can faithfully deliver a trace it has seen, can it *generate* traces for tunnels it has no examples for?

Result. It cannot. None of the three models we tried (Opus, Sonnet, Haiku) wrote a solver that generalized to unseen tunnels, and the detector stays at AUC 1.0. Figure 1 shows the failure mode. Spatially the agent’s paths sit almost exactly on the centerline, which the agent treats as the safest possible route. But staying on the centerline is itself a non-human behavior: the task only requires reaching the goal without crossing a wall, and no human bothers to track the

Agent-generated vs human trajectories on unseen tunnels (split_a)



Speed profile: human vs agent on unseen tunnels (split_a)

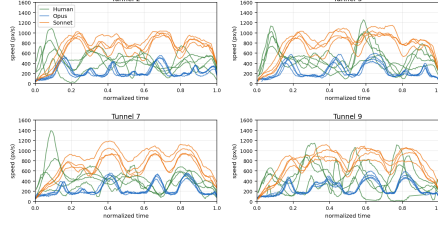


Figure 1: Experiment 2. Spatial paths (left) cluster on the centerline for all three populations, but the speed profiles (right) reveal the gap: agents within a model peak at the same normalized times, and those times do not line up with where humans peak.

centerline so precisely. The speed profiles are where the more interesting gap shows up. Speed has no intrinsic meaning to the agent, since it doesn't get tired and doesn't feel the curvature; it is only trying to mimic what it sees in the example traces. From those examples it correctly picks up the pattern that humans change speed around curves and corners, which is why the *number* of peaks in its speed profile is roughly right. But the *location* of those peaks is wrong: within a model the agent's runs cluster tightly at the same normalized times, and those times don't match where humans peak. The agent can only verify its solver against the example traces it has, and the features it reads off those examples are aggregate statistics, like mean speed, speed standard deviation, and mean centerline deviation. It reproduces those statistics, but it has no insight into when or where in a given tunnel a human would actually accelerate or decelerate to produce them. So the policy is deterministic and matches the summary statistics while getting the per-tunnel timing of acceleration and deceleration wrong. Opus produced the best solvers, Sonnet was weaker, and Haiku could not write a working solver at all, which tracks model capability rather than anything specific to this task.

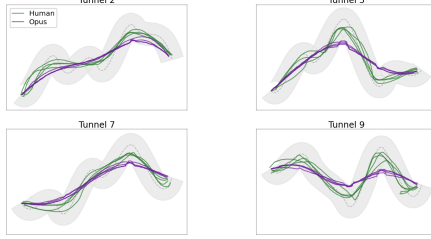
Takeaway. Faithful delivery does not imply faithful generation. The agent has no model of how a human hand would traverse a path it has not been shown, and writing one from a few examples is not within reach of frontier coding agents today.

3.3 Experiment 3: Guided iteration

Question. The agent's early solvers are easily detected. If we actively guide it on how to iterate, can it do meaningfully better?

Result. After roughly thirty minutes of guided adjustment per tunnel, the trajectories look visibly better and the detector falls only slightly, from 1.0 to AUC 0.998. Figure 2 shows the improved solver. Two things change: the speed profile reaches peak amplitudes comparable to humans instead of staying flat,

Improved Opus solver: trajectories on unseen tunnels (split_a)



Improved Opus solver: speed profile on unseen tunnels (split_a)

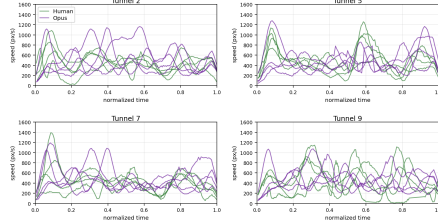


Figure 2: Experiment 3. Guided iteration brings the speed-peak amplitudes close to human, and the path no longer hugs the centerline. But the trajectories now sit closer to the tunnel walls than humans would risk, and the timing of the speed peaks still misses humans.

and the spatial path wanders away from the centerline rather than tracking it. What does not change is the deeper problem, which is that we are still asking the agent to express human behavior as a solver. When we tell it “find a better path,” the agent responds by searching for a better mathematical model of the trajectory: a different waypoint distribution, a different jitter schedule, a different curvature-to-speed mapping. The trouble is that human preference is not a single deterministic policy. A human will take a shortcut when they are feeling lazy but avoid that same shortcut when it looks risky, speed up through one part of a tunnel and crawl through another, push close to a wall on a wide curve and stay far from it on a tight one. None of these choices have a clean natural-language description that the agent can translate into a deterministic function, and the more precise we try to make the description, the more brittle the resulting solver gets. The trajectories the guided solver produces now also graze the tunnel walls in places a human would not, because the agent is optimizing visible trajectory features with no internal cost for the boundary reset humans actually pay attention to. The detector still catches it because the other tells, especially the timing of speed peaks and the missing tremor, remain untouched and the classifier simply re-weights toward them.

Takeaway. Solver-based attacks are structurally bad at expressing human preference, because preference is contextual and non-deterministic and a solver is neither. That is what makes the asymmetry favor the defender so strongly: the adversary pays roughly thirty minutes of guided iteration per tunnel to recover a fraction of a percent of detector AUC, and a defender who makes the geometry more complicated or adds moving obstacles so no static solver can still hold invalidates all of that work for free.

3.4 Experiment 4: RL agent

Question. Can a reinforcement-learning policy trained directly against the task do better than LLM-written solvers?

Result. It does not. The BC+PPO policy reaches AUC 1.0 on held-out tunnels, with the same flat speed profile and missing curve-versus-straight coupling we saw from the LLM solvers. The anti-detection reward narrows the gap on the three features it directly targets and leaves the rest untouched, so the detector switches to weighting those.

Takeaway. The failure is not specific to one technology. Neither an LLM-written solver nor a trained RL policy reproduces human motor behavior on a path it has not seen, which is what the embodiment hypothesis predicts.

3.5 Where the signal lives: which motor features survive

Experiment 1 confirmed that the mechanism head is no longer doing the work once delivery is faithful. Experiments 2 through 4 are about generation, and in all three the detector returns AUC 1.0 from the motor head alone. Looking across the figures, three motor features carry that signal.

Where the speed peaks are. In Experiment 2 (Figure 1), the agent matches the *count* of speed peaks across a tunnel but not their *position*. Every Opus run peaks at the same normalized time; every Sonnet run peaks at the same normalized time; neither of those times is where humans peak. The cause is structural: an LLM-written solver applies one deterministic policy on a given tunnel, so its runs cluster, while a human responds in real time and so their peaks scatter. Guided iteration in Experiment 3 fixes peak *amplitude* but not peak *location*, and the RL policy in Experiment 4 inherits the same cluster behavior.

Risk profile. Humans take paths that stay safely inside the tunnel because crossing the boundary forces a reset. Figure 2 shows the consequence of pushing an agent toward more human-looking features: its trajectories now graze the walls in places a human would never choose to be, because the agent has no internal cost for the reset. Anywhere we ask the agent to imitate a feature of human motion without giving it the constraint humans are responding to, the resulting path is wrong in a way human paths are not.

Continuous-correction noise. Humans produce involuntary 8–12 Hz tremor and small ongoing path corrections; both LLM solvers and the PPO policy produce smooth paths with no power in that band. This is the cleanest tell because there is no source of the noise in a computed path. The anti-detection reward in Experiment 4 closes the gap on the three features it names (centerline deviation, mean speed, path efficiency) and leaves tremor untouched, so the classifier switches to weighting tremor and recovers AUC 1.0.

What the random forest weights. Because the random forest exposes feature importances, we can check whether the classifier is actually reading what we think it is. Against the visual baseline four features carry about two-thirds

of the total importance: centerline-deviation mean (0.23) and standard deviation (0.22), inter-event Δt mean (0.11, the only mechanism-head feature in the top ranks), and speed standard deviation (0.08). Once delivery is faithful and the mechanism head loses its weight (Experiments 2 through 4), the remaining importance moves entirely onto the motor head, with the same families dominating: centerline deviation, speed variability, and 8–12 Hz tremor power. These are the three families the figures above flagged, and the random-forest importance is what tells us the classifier is in fact reading them rather than fitting something subtle we did not name.

These three features are all about the player, not the path. Peak *location* reflects when the player decided to accelerate, risk *tolerance* reflects what the player would choose, and tremor reflects what the player’s body does involuntarily. An agent that knows the path geometry has none of these to draw on, which is why every attack in Experiments 2 through 4 fails on the same head.

3.6 A reasoning-game contrast: the gap exists but the design discards it

To check whether the same human-agent gap appears in a non-motor task, we ran two reasoning games, Pick-the-Personality and Roll-or-Slide, in which the player watches moving shapes and clicks one. Unlike Trace-the-Tunnel, neither game produces a usable trajectory, so we do not train a classifier for them and there is no AUC to report. Each game offers only one type of player action and we measure that action at the outcome level: in Pick-the-Personality the only response is the dot chosen, so the metric is whether the agent picks the right one; in Roll-or-Slide we log how many times each shape is clicked, and treat a session as successful when finished in at most four clicks, since a human who tried each of the four shapes in turn would not need more.

We also exclude reasoning latency from these outcome metrics, and we raise the point here rather than later because latency is the most tempting signal to reach for in a reasoning game. The agent is always slower than a human, but that difference is dominated by execution overhead, dispatching Playwright scripts, capturing screenshots, running analysis code, and generating tokens, rather than by reasoning itself. That overhead is hardware- and harness-dependent and would shift with any change in tooling, so we do not treat latency as a defensible decision factor.

On the outcome metrics that remain, the gap is small or absent. In Pick-the-Personality, the agent can start low when given only the target label in isolation (e.g. “find the FOLLOWER”) and confuse semantically adjacent personalities such as lazy with follower; once it has seen each personality in motion at least once, it builds a numerical signature per label and its accuracy approaches 100%. In Roll-or-Slide the agent stays within the four-click threshold, indistinguishable from humans on the click-count metric. So on the outcome channel both populations look the same.

The reason the gap does not surface is structural. A human glances for a few seconds, perceives the answer through fast System-1 pattern-matching,

Table 2: Single-commit reasoning game family. Four objects move on a canvas; the player watches, reasons, and commits with one click.

Ver.	Title	Objects	Rule difference	How player observes
v1	Spot the Different One	Irregular polygons	Roll vs. slide	Click shape to trigger preview sweep
v2	Spot the Different One	Squares	Roll vs. slide	Click shape to trigger preview sweep
v3	Spot the Different One	Circles	Roll vs. slide	Click shape to trigger preview sweep
v4	Race Watcher	Horizontal racers	Kinematic profile (ease-in, ease-out, pause, reverse)	Watch live animation
v5	Physics Room	Bouncing balls	Physics rule (floater, heavy, hyper, drifter)	Watch live physics simulation
v6	Pick the Personality	Colored dots	Behavioral personality (leader, follower, hyper, lazy)	Watch 10s live animation; click named personality

and clicks. The agent instead captures roughly twelve seconds of canvas at 80 ms intervals, computes per-dot centroids and lagged cross-correlations into a numerical signature, and then clicks. The two processes are completely different, but a single-commit design records only the final click and discards everything between observation and answer. The lesson is a design principle, that the discriminating signal must be a necessary byproduct of completing the task. The tunnel forces a continuous motor record; in the reasoning game the cursor sits idle during analysis, so the same trajectory features see nothing. The table lists the 6 variations that we created; on each of them the agent’s accuracy and click count are comparable to a human’s.

3.7 Summary

Motor games show real potential for separating humans from both LLM coding agents and RL agents: in every condition where the agent had not seen the exact path before, the trajectory classifier reached AUC 1.0, whether the attacker was an LLM-written solver, a human-guided iteration of it, or a PPO policy trained on the task.

The reasoning-game contrast adds a second lesson: a game must be built so that intermediate decisions are always collected, not just the final answer. The motor game discriminates because the cursor stream is a necessary byproduct of completing the task; Pick-the-Personality and Roll-or-Slide do not, because their single-commit click discards everything between observation and answer,

including the screenshot-and-cross-correlation pipeline that makes the agent’s process categorically different.

Limitations and scope. This is a systems study with a deliberately narrow scope. The human data are small (217 sessions from six participants on ten tunnels), so the classifier may partly fit our participants rather than humans in general (participants are six Stanford computer-science students recruited from the authors’ immediate network; all played on personal MacBooks with built-in trackpads or external mice in Chromium, and we did not vary device type, OS, or input modality, so the human pool is demographically and hardware-homogeneous and the classifier may partly fit this setting). Our adversary is an LLM coding agent plus one RL policy; it is not a vision-language agent that perceives the rendered game end-to-end, so we scope every conclusion to LLM and RL agents, and VLM agents remain the most relevant untested threat. Finally, the clean measurement depends on a fixed tunnel pool, which is also what an adversary can memorize, so rotating or expanding the pool trades measurement cleanliness for robustness.

4 Team Responsibilities

All work was split among us equally.

5 References

References

- Johnny Accot and Shumin Zhai. Beyond fitts’ law: Models for trajectory-based HCI tasks. In *Proceedings of the ACM SIGCHI Conference on Human Factors in Computing Systems (CHI)*, pages 295–302, 1997.
- Alejandro Acien, Aythami Morales, Julian Fierrez, and Ruben Vera-Rodriguez. BeCAPTCHA-Mouse: Synthetic mouse trajectories and improved bot detection. *Pattern Recognition*, 127:108643, 2022.
- Ismail Akrouit, Amal Feriani, and Mohamed Akrouit. Hacking google reCAPTCHA v3 using reinforcement learning. *arXiv preprint arXiv:1903.01003*, 2019.
- Ziqi Ding, Gelei Deng, Yi Liu, Junchen Ding, Jieshan Chen, Yulei Sui, and Yuekang Li. IllusionCAPTCHA: A CAPTCHA based on visual illusion. *arXiv preprint arXiv:2502.05461*, 2025.
- Rodger J. Elble and William C. Koller. *Tremor*. Johns Hopkins University Press, 1990.

- Paul M. Fitts. The information capacity of the human motor system in controlling the amplitude of movement. *Journal of Experimental Psychology*, 47(6):381–391, 1954.
- Tamar Flash and Neville Hogan. The coordination of arm movements: An experimentally confirmed mathematical model. *Journal of Neuroscience*, 5(7):1688–1703, 1985.
- Fritz Heider and Marianne Simmel. An experimental study of apparent behavior. *American Journal of Psychology*, 57(2):243–259, 1944.
- Imperva. 2025 imperva bad bot report: The rapid rise of bots and the unseen risk for business. Technical report, Imperva, Inc. (Thales), 2025.
- Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.
- Simon Khan and Daqing Hou. Mouse dynamics behavioral biometrics: A survey. *ACM Computing Surveys*, 2024. doi: 10.1145/3640311.
- Arina Kharlamova, Bowei He, Chen Ma, and Xue Liu. Spatial CAPTCHA: Generatively benchmarking spatial reasoning for human-machine differentiation. *arXiv preprint arXiv:2510.03863*, 2025.
- Jiacheng Liu, Yaxin Luo, Jiacheng Cui, Xinyi Shang, Xiaohan Zhao, and Zhiqiang Shen. Next-gen captchas: Leveraging the cognitive gap for scalable and diverse gui-agent defense. *arXiv preprint arXiv:2602.09012*, 2026.
- Jiahua Liu, Zeyuan Cui, Wenhan Ge, and Pengxiang Zhan. Dmtg: A human-like mouse trajectory generation bot based on entropy-controlled diffusion networks. *arXiv preprint arXiv:2410.18233*, 2024.
- Yaxin Luo, Zhaoyi Li, Jiacheng Liu, Jiacheng Cui, Xiaohan Zhao, and Zhiqiang Shen. Open CaptchaWorld: A comprehensive web-based platform for testing and benchmarking multimodal LLM agents. *arXiv preprint arXiv:2505.24878*, 2025.
- Andreas Plesner, Tobias Vontobel, and Roger Wattenhofer. Breaking recaptchav2. In *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*, pages 1047–1056. IEEE, 2024.
- Brian J. Scholl and Patrice D. Tremoulet. Perceptual causality and animacy. *Trends in Cognitive Sciences*, 4(8):299–309, 2000.
- Chao Shen, Zhongmin Cai, Xiaohong Guan, Youtian Du, and Roy A. Maxion. User authentication through mouse dynamics. *IEEE Transactions on Information Forensics and Security*, 8(1):16–30, 2013.
- Xiwen Teoh, Yun Lin, Siqi Li, Ruofan Liu, Avi Sollomoni, Yaniv Harel, and Jin Song Dong. Are CAPTCHAs still bot-hard? generalized visual CAPTCHA solving with agentic vision language model. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 3747–3766, 2025.

Luis Von Ahn, Manuel Blum, Nicholas J. Hopper, and John Langford.
CAPTCHA: Using hard AI problems for security. In *International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, pages 294–311. Springer, 2003.